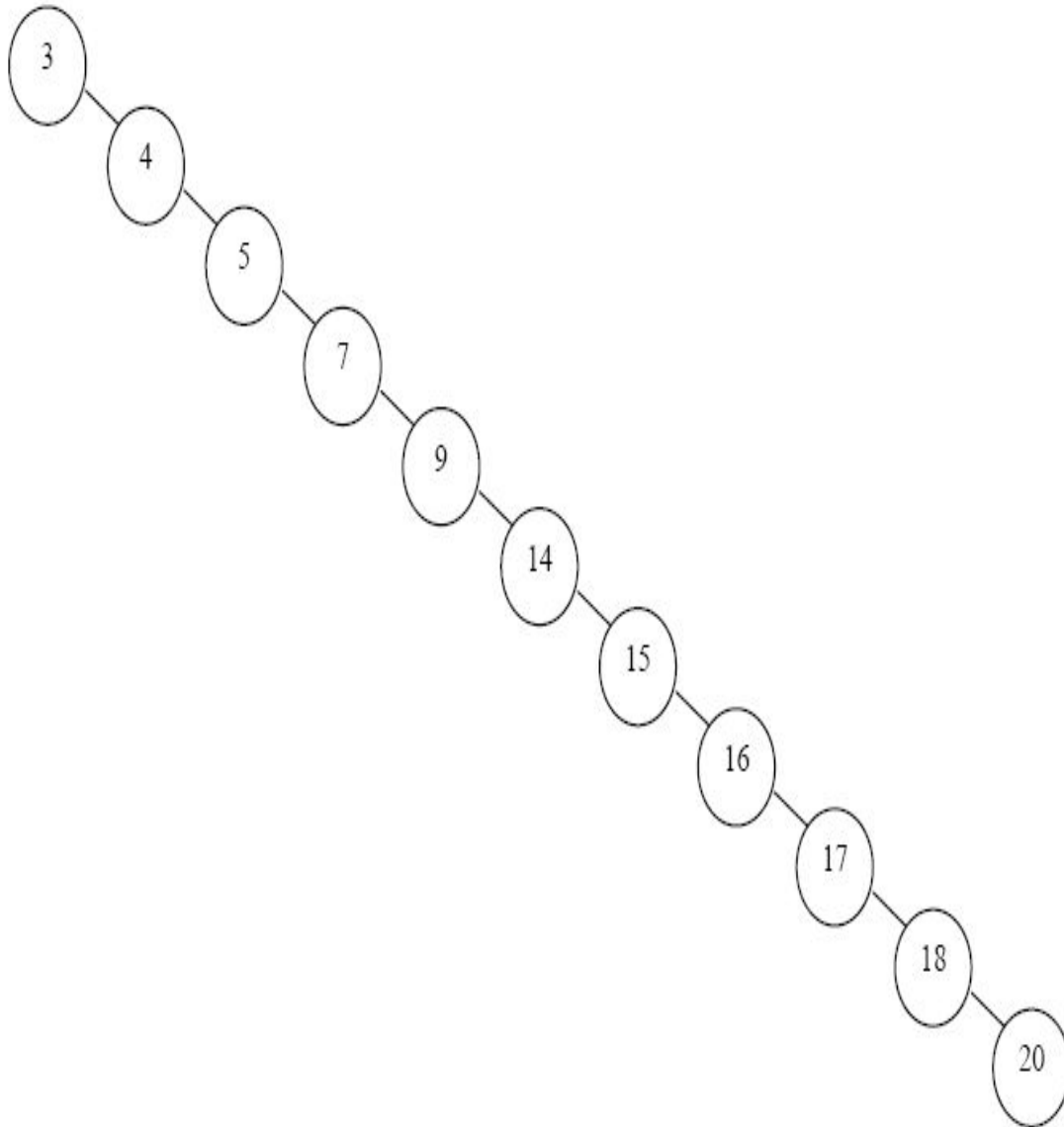


BST and AVL

DATA STRUCTURE

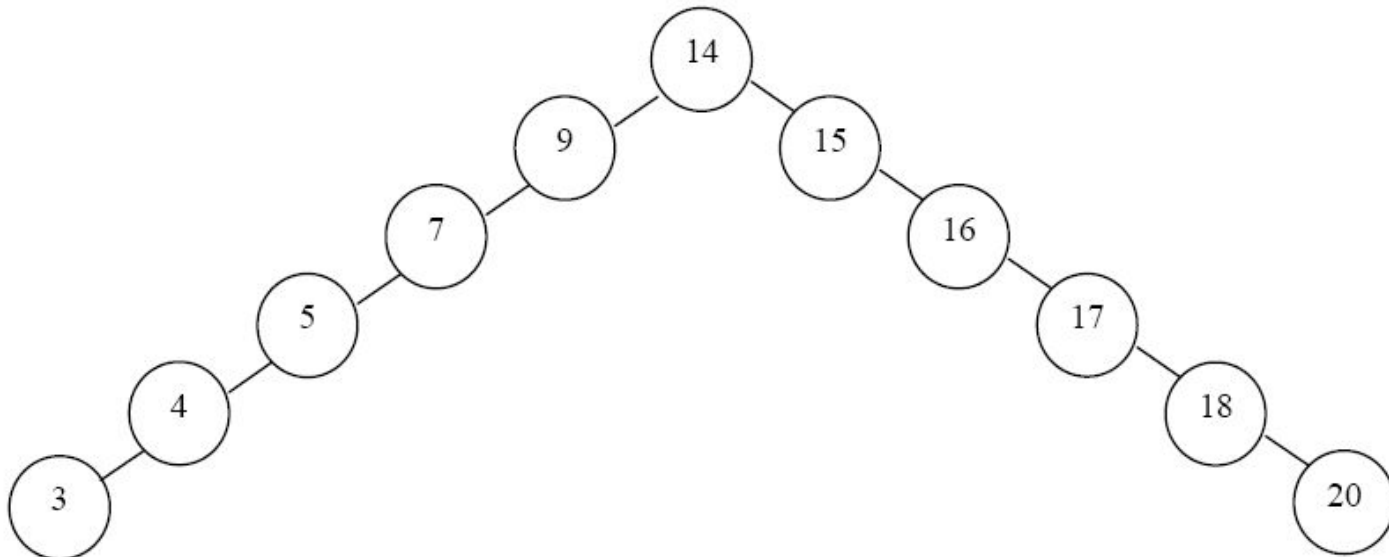
BST for 3, 4, 5, 7, 9, 14, 15, 16, 17, 18, 20



Degenerate
BST

One idea to Solve

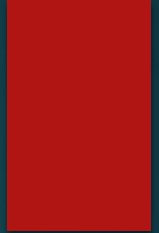
- ❑ We should keep the tree balanced
 - ❑ One idea would be to have the left and right subtrees have the same height.
- ❑ Does not force the tree to be **shallow**



Second Way of Solution

- ❑ We could insist that every node must have left and right subtrees of the same height.
- ❑ But this requires that the tree be a complete binary tree
- ❑ To do this, we must have $(2^{d+1} - 1)$ data item, where d is the depth of the tree.

AVL Tree



- ❑ AVL (Adelson-Velskii and Landis)
- ❑ An AVL tree is identical to a BST except
 - ❑ Height of the left and right subtrees can differ by at most 1.
 - ❑ Height of an empty tree is defined to be (-1).
- ❑ Remember that this condition is not at the root. It should satisfy at any level at any node.

AVL Tree:

- ❑ **Height**—The height of a binary tree is the maximum level of its leaves. This is the same definition as of depth of a tree.
- ❑ The height of a tree is the longest path from the root to the leaf. This can also be calculated as the maximum level of the tree.
- ❑ If we have to calculate the height of some node, we should start counting the levels from that node.

Balance

- ❑ The balance of a node in a binary search tree is defined as the height of its left subtree minus height of its right subtree.
- ❑ In other words, at a particular node, the difference in heights of its left and right subtree gives the balance of the node.
- ❑ Each node has a balance 1, 0 or -1 .

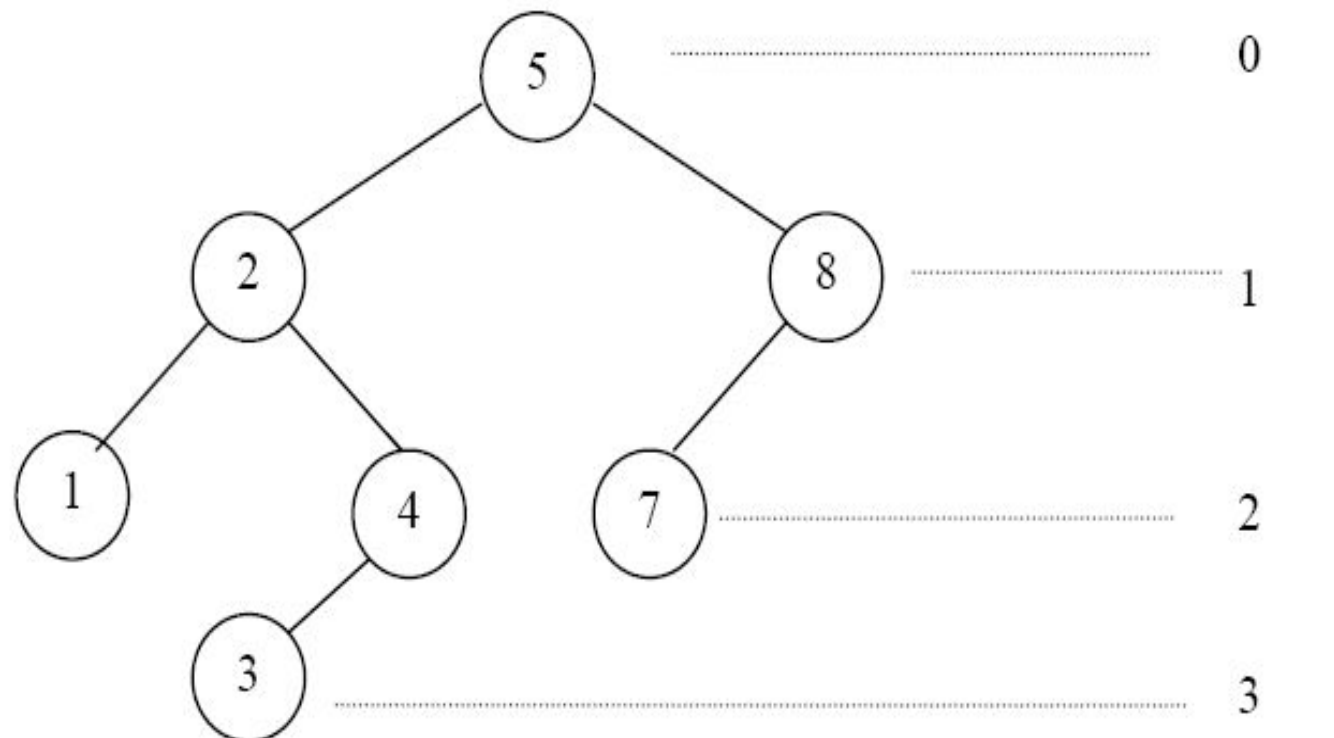
AVL Tree

- ❑ The condition in the AVL tree is that at any node the height of the left subtree can be one more or one less than the height of right subtree.
- ❑ The height can be equal.
- ❑ The difference of heights cannot be more than 1.
- ❑ This can be 1,-1 or 0.
- ❑ This AVL definition does not apply only to the root node of the tree. Every node (non-leaf or leaf) should fulfill this definition. This means that the balance of every node should be 1, 0 or -1 . Otherwise, it will not be an AVL tree.

Balance formula

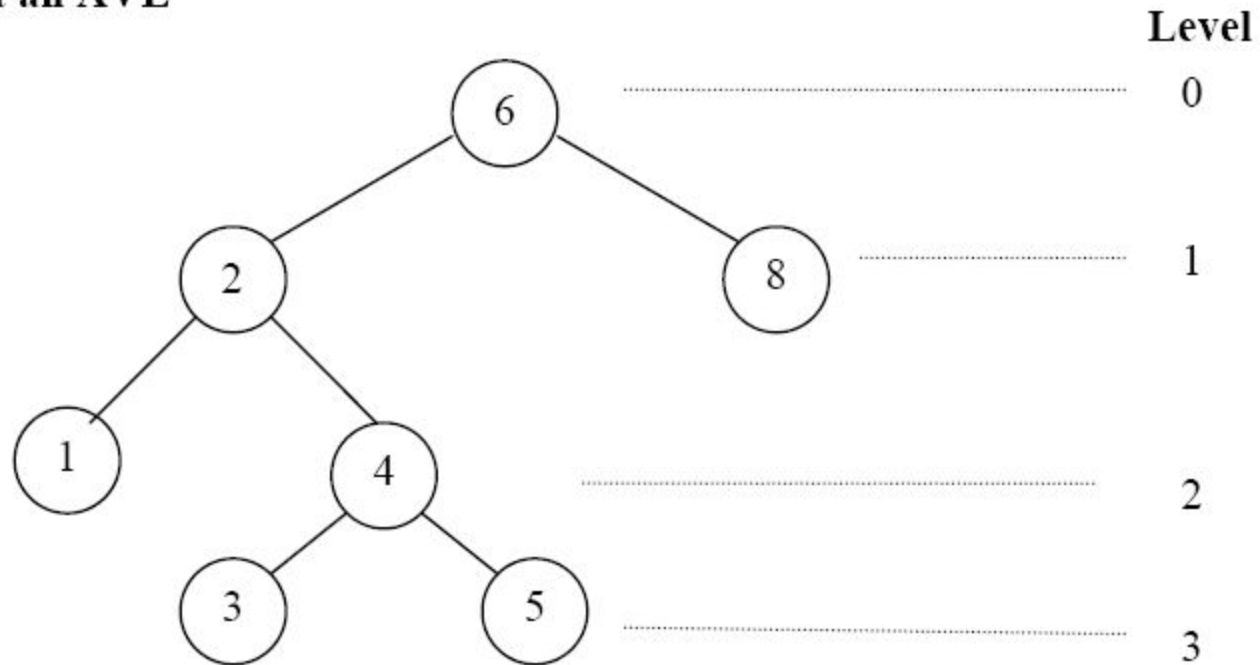
Balance factor= left Height - Right height

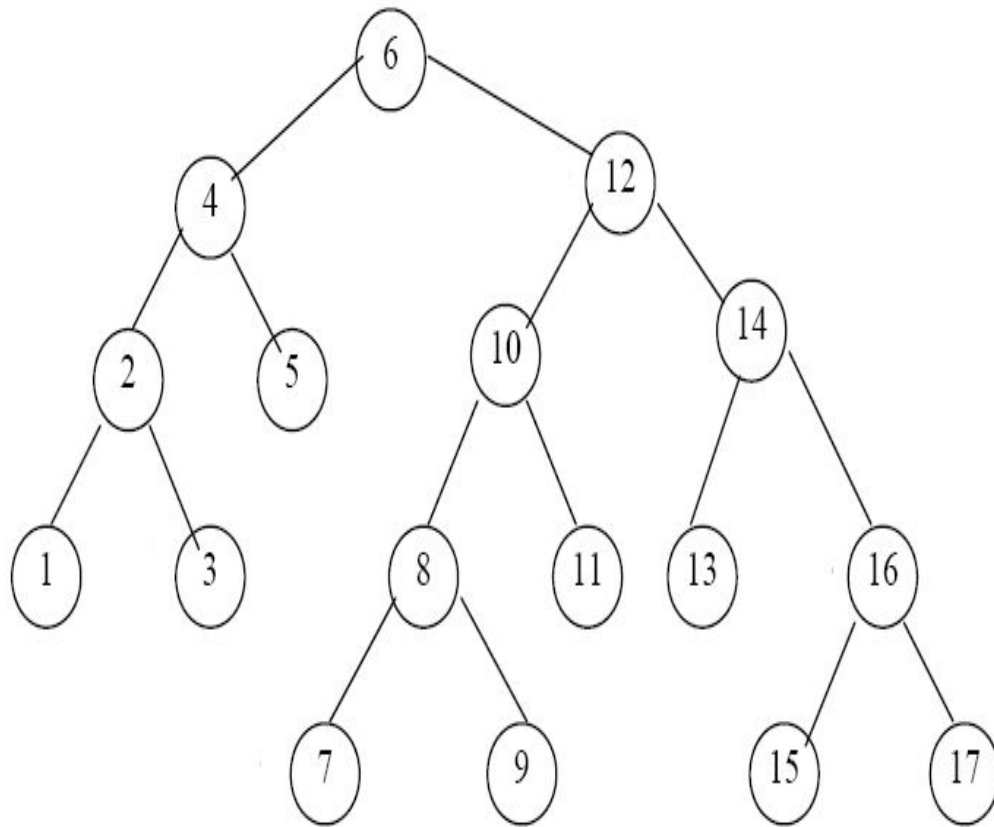
An AVL Tree



Not an AVL Tree

- Not an AVL



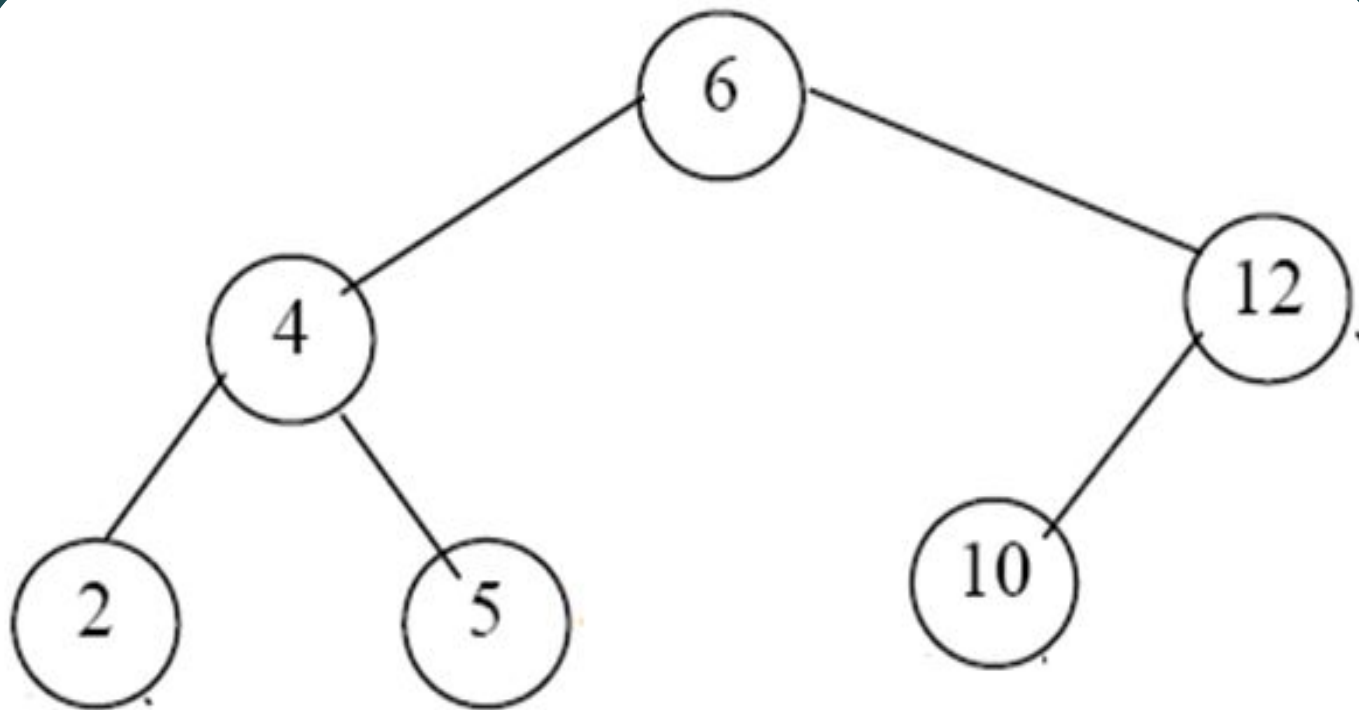


Check Condition of an AVL tree

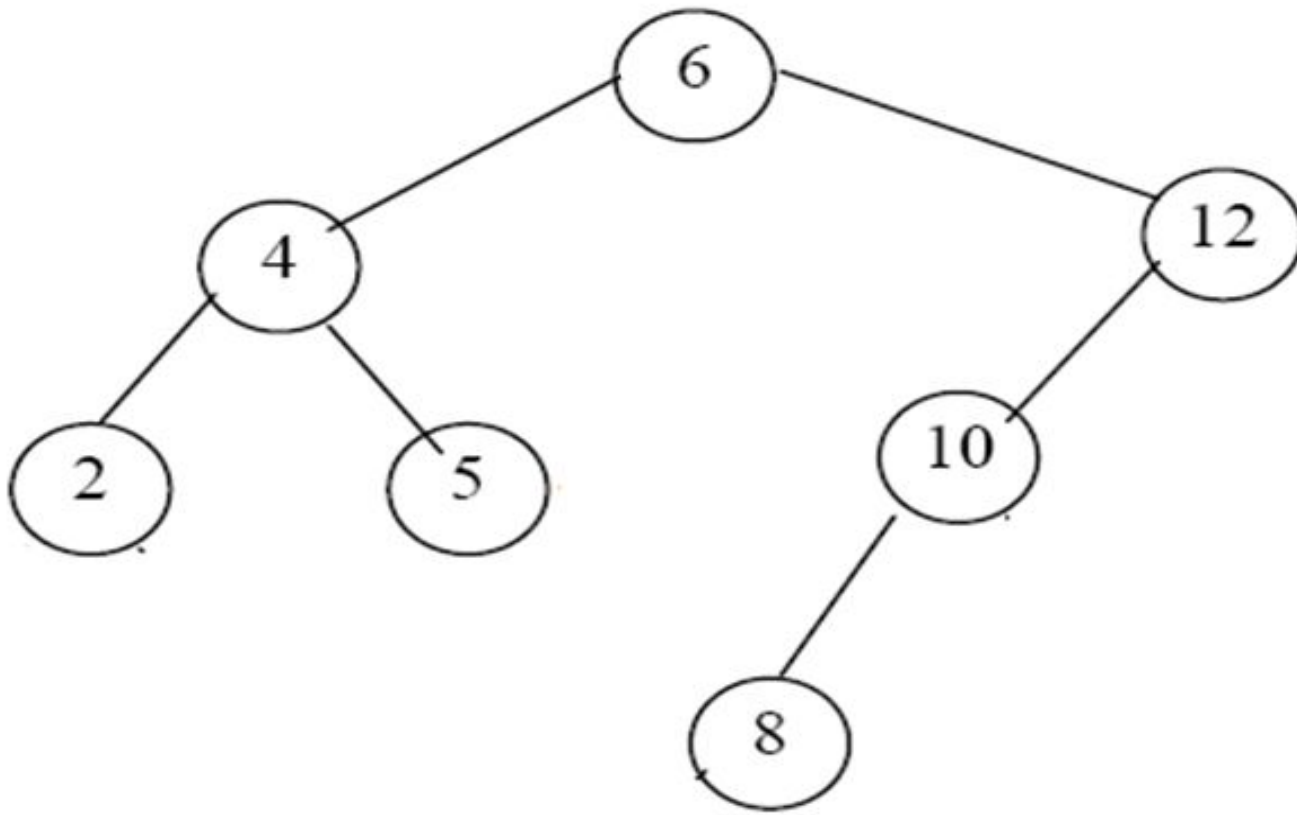
Insertion of Node in AVL Tree

AVL TREE BUILDING

Insertion of a Node in AVL Tree:



Insertion of a Node in AVL Tree:



Rotation Cases:

- □ (node requires to be re-balanced).
- A height imbalance requires that α 's two sub-trees differ by two (or -2)
 1. An insertion into left subtree of the left child of □.
 2. An insertion into right subtree of the left child of □.
 3. An insertion into left subtree of the right child of □.
 4. An insertion into right subtree of the right child of □.

Rotation Cases:

1. An insertion into left subtree of the left child of $\square$. (**Single Right rotate**)
2. An insertion into right subtree of the left child of $\square$. (**double left-right rotate**)
3. An insertion into left subtree of the right child of $\square$. (**double right-left rotate**)
4. An insertion into right subtree of the right child of $\square$. (**Single left rotate**)

2. An insertion into right subtree of the left child of $\square$. (**double left-right rotate**)

3. An insertion into left subtree of the right child of $\square$. (**double right-left rotate**)

63,9,19,27,18,108,99,81

